

### Binary Tree Paths :

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<String> binaryTreePaths(TreeNode root) {
        List<String> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> q = new LinkedList<>();
        Queue<String> paths = new LinkedList<>();
        q.add(root);
        paths.add(String.valueOf(root.val));
        while (!q.isEmpty()) {
            TreeNode curr = q.poll();
            String path = paths.poll();

            if (curr.left == null && curr.right == null) result.add(path);

            if (curr.left != null) {
                q.add(curr.left);
                paths.add(path + "->" + curr.left.val);
            }

            if (curr.right != null) {
                q.add(curr.right);
                paths.add(path + "->" + curr.right.val);
            }
        }
        return result;
    }
}
```

## Binary Tree Right Side View :

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        if (root == null) return new ArrayList<>();

        ArrayList<ArrayList<Integer>> ans = new ArrayList<>();
        Queue<TreeNode> q = new LinkedList<>();
        q.add(root);

        while (!q.isEmpty()){
            int n = q.size();
            ArrayList<Integer> al = new ArrayList<>();

            for (int i = 0; i < n; i++){
                TreeNode curr = q.peek();
                if (curr != null){
                    if (curr.left != null) q.add(curr.left);
                    if (curr.right != null) q.add(curr.right);
                    al.add(curr.val);
                    q.poll();
                }
            }
            ans.add(al);
        }

        List<Integer> result = new ArrayList<>();
        for (ArrayList<Integer> i: ans){
            int len = i.size() - 1;
            result.add(i.get(len));
        }
    }
}
```

```

        return result;
    }
}

```

Construct Binary Tree from Preorder and Inorder Traversal :

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        TreeNode root = null;
        Map<Integer, Integer> map = new HashMap<Integer, Integer>();

        for (int i=0; i<inorder.length; i++){
            map.put(inorder[i], i);
        }

        return constructTree(preorder, inorder, map, 0, inorder.length - 1);
    }

    int i = 0;
    public TreeNode constructTree(int[] preorder, int[] inorder, Map<Integer, Integer> map, int
start, int end){
        if (start > end) return null;

        TreeNode newNode = new TreeNode(preorder[i]);
        int index = map.get(preorder[i]);
        i += 1;
        newNode.left = constructTree(preorder, inorder, map, start, index - 1);
        newNode.right = constructTree(preorder, inorder, map, index + 1, end);
        return newNode;
    }
}

```

Left View of Binary Tree :

/\* Structure of Binary Tree Node

```
class Node {
    int data;
    Node left, right;

    Node(int val) {
        this.data = val;
        this.left = this.right = null;
    }
}*/
```

```
class Solution {
    public ArrayList<Integer> leftView(Node root) {
        ArrayList<Integer> result = new ArrayList<>();
        if (root == null) return result;
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        while (!q.isEmpty()) {
            int n = q.size();
            for (int i = 0; i < n; i++) {
                Node curr = q.poll();
                if (i == 0) result.add(curr.data);
                if (curr.left != null) q.add(curr.left);
                if (curr.right != null) q.add(curr.right);
            }
        }
        return result;
    }
}
```

Lowest Common Ancestor of a Binary Tree :

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
```

```

class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode
q) {
        if (root == null || root == p || root == q) {
            return root;
        }

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

        if (left != null && right != null) {
            return root;
        }

        return left != null ? left : right;
    }
}

```

Tree Boundary Traversal :

```

/*
class Node {
    int data;
    Node left, right;

    public Node(int d) {
        data = d;
        left = right = null;
    }
}
*/

class Solution {
    public void leftfun(Node root, ArrayList<Integer> a){
        if (root == null) return;
        if (root.left == null && root.right == null) return;

        if (root.left != null){
            a.add(root.data);
            leftfun(root.left, a);
        } else {
            a.add(root.data);
            leftfun(root.right, a);
        }
    }
}

```

```

public void rightfun(Node root, ArrayList<Integer> a){
    if (root == null) return;
    if (root.left == null && root.right == null) return;

    if (root.right != null) rightfun(root.right, a);
    else rightfun(root.left, a);

    a.add(root.data);
}

public void leaffun(Node root, ArrayList<Integer> a){
    if (root == null) return;
    if (root.left == null && root.right == null) a.add(root.data);
    leaffun(root.left, a);
    leaffun(root.right, a);
}

ArrayList<Integer> boundaryTraversal(Node root) {
    // code here
    ArrayList<Integer> a = new ArrayList<>();
    a.add(root.data);
    leftfun(root.left, a);
    leaffun(root.left, a);
    leaffun(root.right, a);
    rightfun(root.right, a);
    return a;
}
}

```